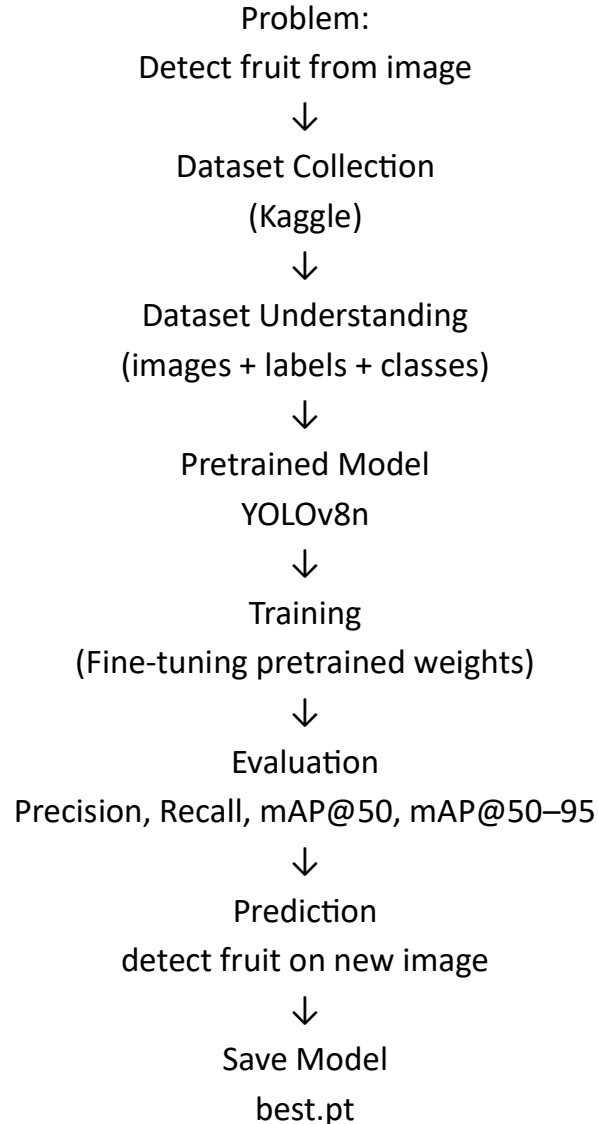
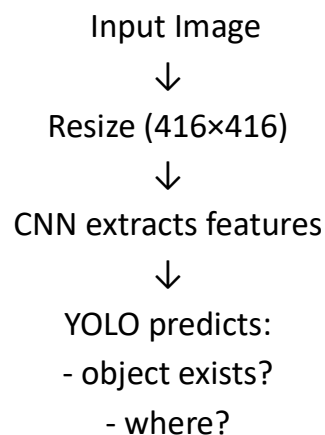


Fruit Detection using YOLOv8 and Deep Learning

Complete Pipeline



What happened internally



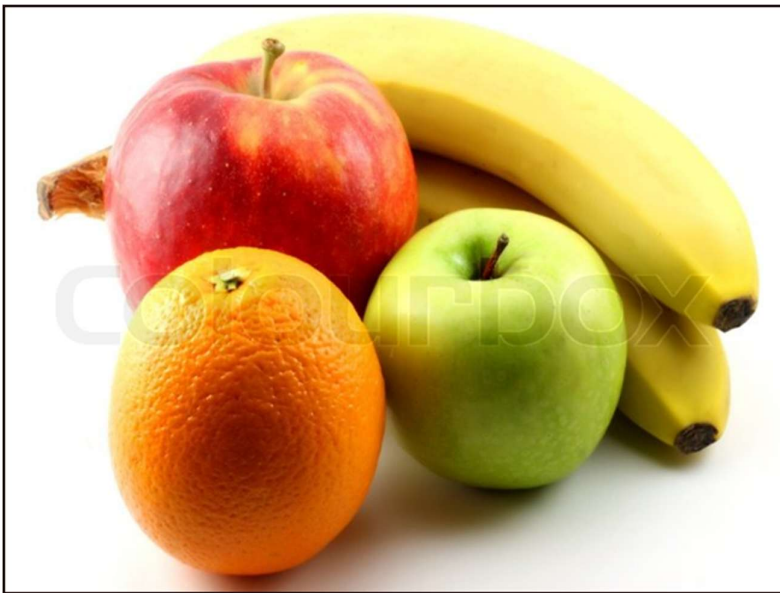
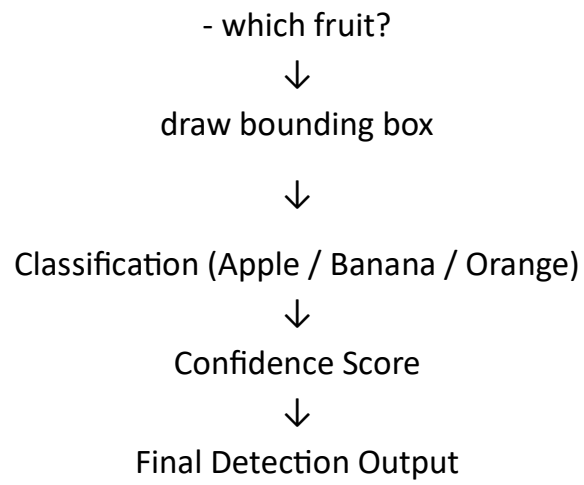


Figure: Sample image from dataset.

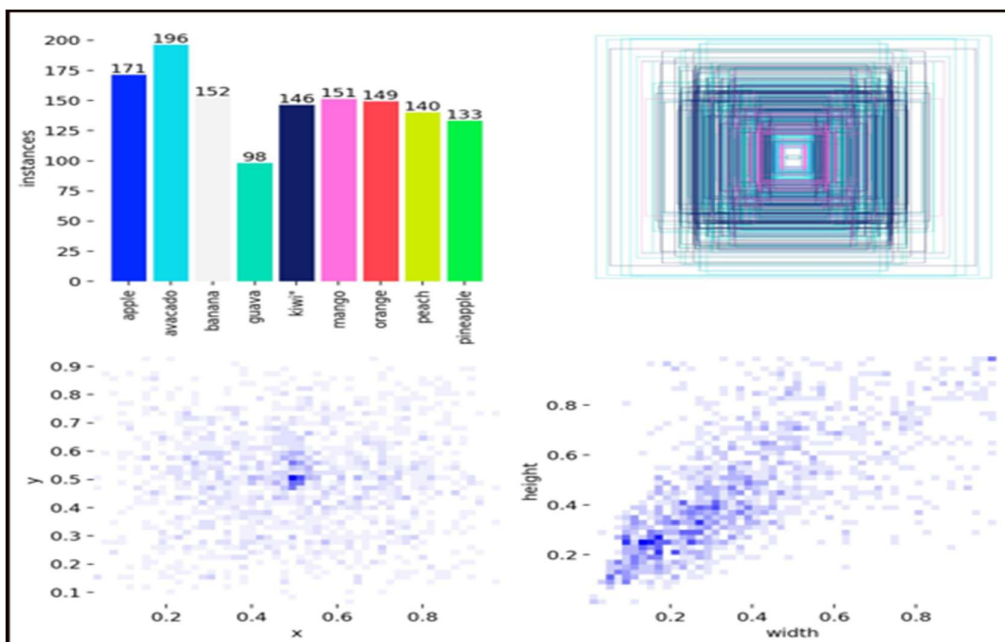


Figure: Distribution of fruit labels in dataset.

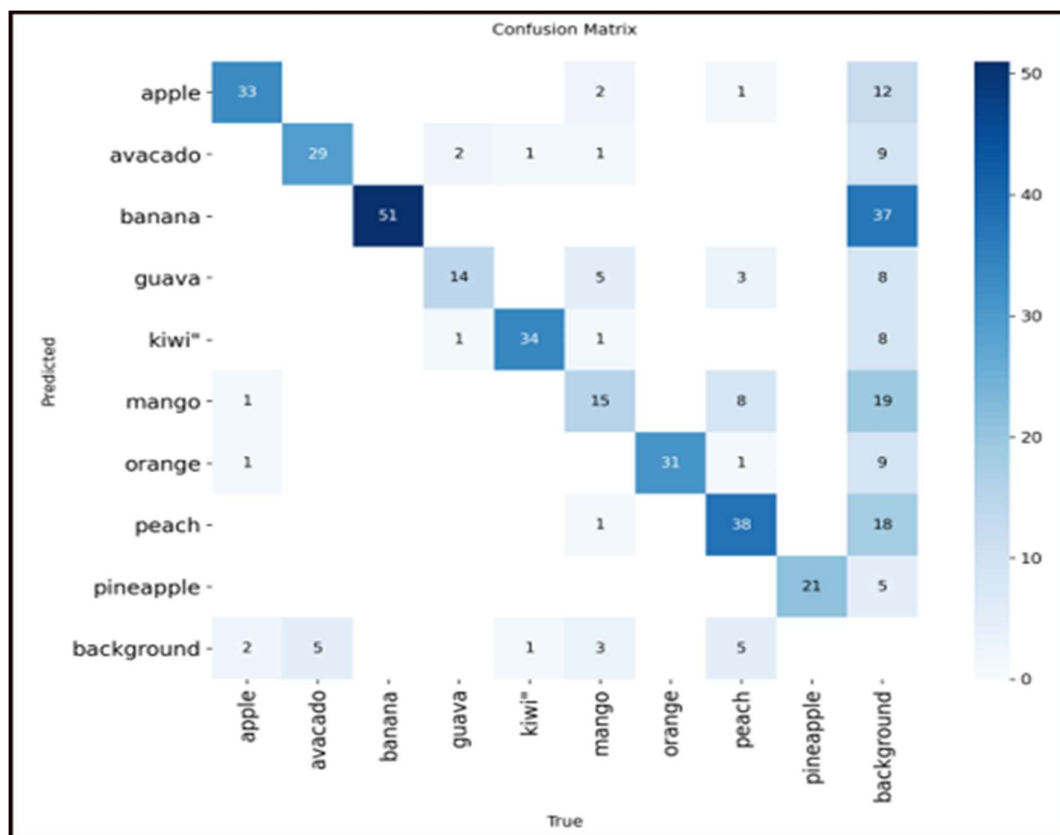


Figure: Confusion matrix showing classification performance.



Figure: Prediction result using trained YOLOv8 model.

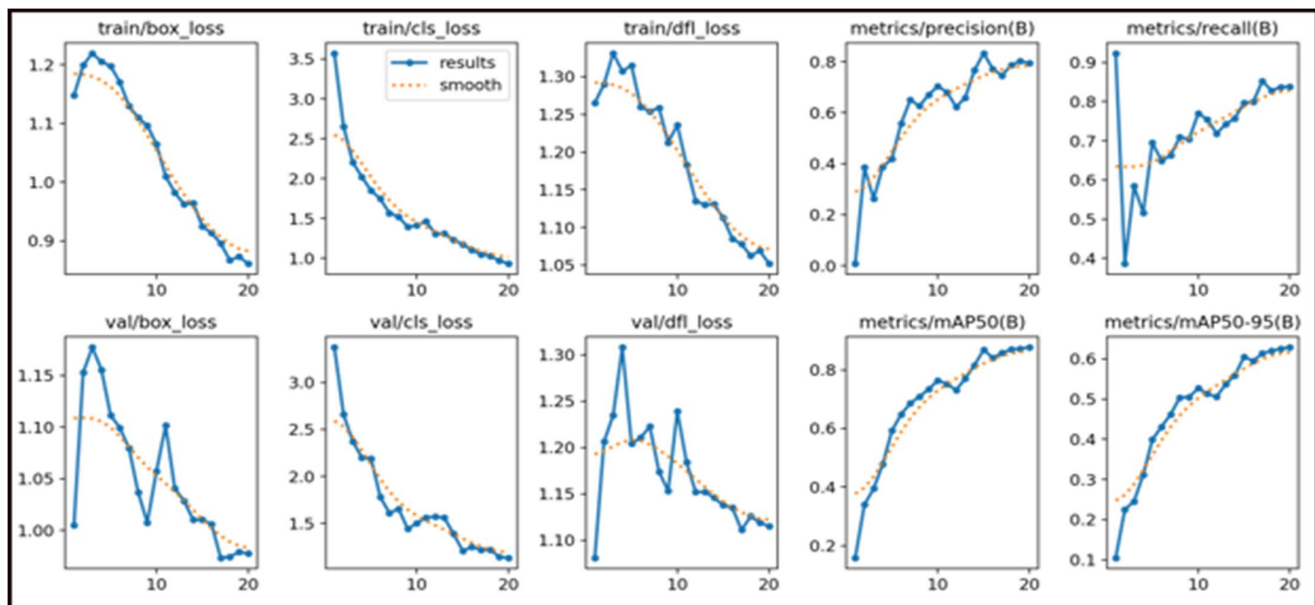


Figure: Training and validation performance across epochs.

1. Introduction

Object detection is a computer vision task that identifies objects in an image and locates them using bounding boxes. Unlike image classification, object detection can detect multiple objects and identify their positions. In this project, a fruit detection system was developed using the YOLOv8 deep learning model.

The objective of this project is to detect and classify fruits from images automatically.

2. Objective

The main objectives of this project are:

- To understand the object detection workflow.
- To collect and prepare a fruit image dataset.
- To train a pretrained YOLOv8 model.
- To evaluate model performance.
- To predict fruits from new images.

3. Dataset Collection

The dataset was collected from Kaggle.

Dataset details:

- Type: Object Detection Dataset
- Classes: 9

- Categories:
 - Apple
 - Avacado
 - Banana
 - Guava
 - Kiwi
 - Mango
 - Orange
 - Peach
 - Pineapple

Dataset structure:

- images/train
- images/val
- labels/train
- labels/val
- data.yaml

Dataset size:

- Training images: 534
- Validation images: 134

4. Data Understanding and Preprocessing

Before training, the dataset structure and images were inspected.

Steps performed:

- Loaded dataset using Python.
- Read dataset configuration from data.yaml.
- Verified image and label folders.
- Displayed sample images.
- Counted training and validation images.

YOLO automatically performed:

- Image resizing
- Data loading

- Batch processing
- Internal augmentation

Training image size:

- imgsz = 416
-

5. Model Selection

A pretrained YOLOv8 Nano model (yolov8n.pt) was selected.

Reason for choosing YOLOv8:

- Faster training
- Suitable for object detection
- Supports transfer learning
- Easy deployment

Transfer learning was used because pretrained weights already learned general image features.

6. Model Training

The model was trained using:

- Epochs: 20
- Image Size: 416
- Framework: Ultralytics YOLO

Training Process:

Input Image

→ Feature Extraction

→ Object Detection

→ Classification

→ Bounding Box Prediction

7. Evaluation Metrics

The model performance was evaluated using:

- Precision
- Recall
- mAP50 (Good detection)

- mAP50–95 (Bounding box precision)

Observed results:

- Precision ≈ 0.79
- Recall ≈ 0.83
- mAP50 ≈ 0.87
- mAP50–95 ≈ 0.62

The results indicate that the model learned to detect fruit objects successfully.

8. Prediction and Testing

The trained model was tested on:

- Validation images
- New custom fruit images
- Mixed fruit images

Prediction output:

- Fruit class
- Confidence score
- Bounding box

The model successfully detected multiple fruits in a single image.

9. Deployment Concept

A simple deployment approach was explored:

Mobile Camera

→ Capture Image

→ Send Image to System

→ Load Trained Model

→ Predict Fruit

Future improvements:

- Mobile application deployment
 - Real-time camera detection
 - TensorFlow Lite conversion
-

10. Conclusion

This project demonstrated the complete deep learning workflow for object detection using YOLOv8. The project covered dataset preparation, preprocessing, transfer learning, model training, evaluation, prediction, and basic deployment concepts.

The model achieved good detection performance and successfully identified fruits from unseen images.
